

INFRAESTRUTURA COMPUTACIONAL

CST EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Prof Francisco Martins

Material cedido pelo prof. Roberto Cordeiro Waltz

UNIP

UNIVERSIDADE PAULISTA

INFRAESTRUTURA COMPUTACIONAL

CONTEÚDO PROGRAMÁTICO

- **Conceituação e contextualização**
- **Componentes básicos**
- **Gerenciamento de memória**
- **Sistemas de numeração**
- **Algumas aplicações de segurança**
- **Fundamentos de sistemas operacionais**
- **Fundamentos de redes**

INFRAESTRUTURA COMPUTACIONAL

BIBLIOGRAFIA

BÁSICA

STALLINGS, W. Arquitetura e organização de computadores. 10. ed. São Paulo: Prentice Hall, 2017.

SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. Fundamentos de sistemas operacionais. 9. ed. São Paulo: LTC, 2015.

TANENBAUM, Andrew S.; BOS, Hebert. Sistemas operacionais modernos. 4. ed. São Paulo: Pearson Education do Brasil, 2016.



UNIVERSIDADE PAULISTA

BIBLIOGRAFIA BÁSICA

HENNESSY, John. Arquitetura de Computadores - Uma Abordagem Quantitativa. Grupo GEN, 2019. 9788535291759.

<https://integrada.minhabiblioteca.com.br/#/books/9788535291759/>.

STALLINGS, W. Arquitetura e organização de computadores. 10. ed. São Paulo: Prentice Hall, 2017.

<https://plataforma.bvirtual.com.br/Leitor/Publicacao/151479/pdf/0>

JOSÉ, DELGADO,; CARLOS, RIBEIRO,. Arquitetura de Computadores, 5ª edição. Grupo GEN, 2017. 9788521633914.

<https://integrada.minhabiblioteca.com.br/#/books/9788521633914/>.

INFRAESTRUTURA COMPUTACIONAL

BIBLIOGRAFIA

COMPLEMENTAR

CARVALHO, André C. P. L. de, LORENA, Ana Carolina. Introdução à Computação: Hardware, Software e Dados. Rio de Janeiro: LTC, 2017.

CORREA, A. G. D. Organização e arquitetura de computadores. São Paulo: Pearson Education do Brasil, 2016.

PAIXÃO, Renato R. Arquitetura de Computadores. São Paulo: Erica, 2014.

MACHADO, F. B.; MAIA, L. P. Fundamentos de sistemas operacionais. 5ª edição. Rio de Janeiro: LTC, 2017.

Stuart Brian. Princípios de sistemas de operacionais: Projetos e aplicações. São Paulo, Cengage Learning, 2010.

BIBLIOGRAFIA

COMPLEMENTAR

J., FLYNN, M.; WAYNE, LUK,. Projeto de Sistemas de Computador - System-on-Chip. Grupo GEN, 2014. 9788521627234.

<https://integrada.minhabiblioteca.com.br/#/books/9788521627234/>

HENNESSY, John. Organização e Projeto de Computadores. Grupo GEN, 2017. 9788595152908.

<https://integrada.minhabiblioteca.com.br/#/books/9788595152908/>

WEBER, Raul F. Fundamentos de Arquitetura de Computadores - V8 - UFRGS. Grupo A, 2012. 9788540701434.

<https://integrada.minhabiblioteca.com.br/#/books/9788540701434/>

BIBLIOGRAFIA

COMPLEMENTAR

MONTEIRO, Mario A. Introdução à Organização de Computadores, 5ª edição. Grupo GEN, 2007. 978-85-216-1973-4.

<https://integrada.minhabiblioteca.com.br/#/books/978-85-216-1973-4/>.

IRV, ENGLANDER,. A Arquitetura de Hardware Computacional, Software de Sistema e Comunicação em Rede, 4ª edição. Grupo GEN, 2011. 9788521618690.

<https://integrada.minhabiblioteca.com.br/#/books/9788521618690/>.

INFRAESTRUTURA COMPUTACIONAL

AVALIAÇÃO

Avaliação individual (100%)



UNIVERSIDADE PAULISTA

INFRAESTRUTURA COMPUTACIONAL

ESTUDOS DISCIPLINARES

Escolher livremente 10 (dez) exercícios na plataforma.

Prazo: informado no acesso ao ambiente.

INFRAESTRUTURA COMPUTACIONAL

Organização de computadores

Refere-se às unidades operacionais e suas interconexões que implementam as especificações da sua arquitetura:

- **Sinais de controle**
- **Interfaces com periféricos**
- **Tecnologia de memória.**
- **Sistemas computacionais**

INFRAESTRUTURA COMPUTACIONAL

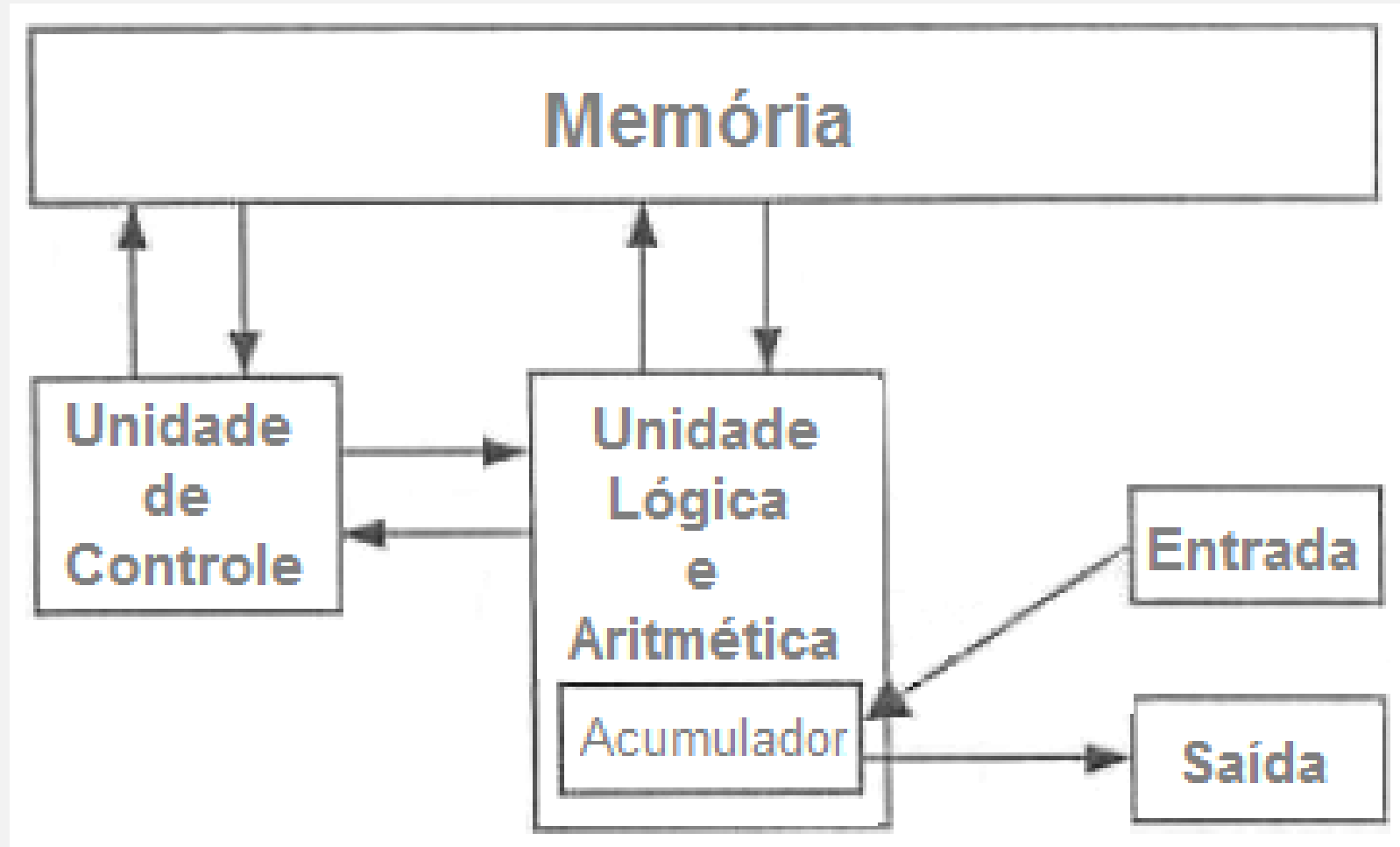
Arquitetura de computadores

Refere-se aos atributos de um sistema que são visíveis para o programador, ou seja, que têm impacto direto sobre a execução lógica de um programa:

- **Conjunto de instruções**
- **Número de bits usados para representar dados**
- **Mecanismos de E/S**
- **Técnicas de endereçamento de memória**

INFRAESTRUTURA COMPUTACIONAL

Componentes básicos de um computador



INFRAESTRUTURA COMPUTACIONAL

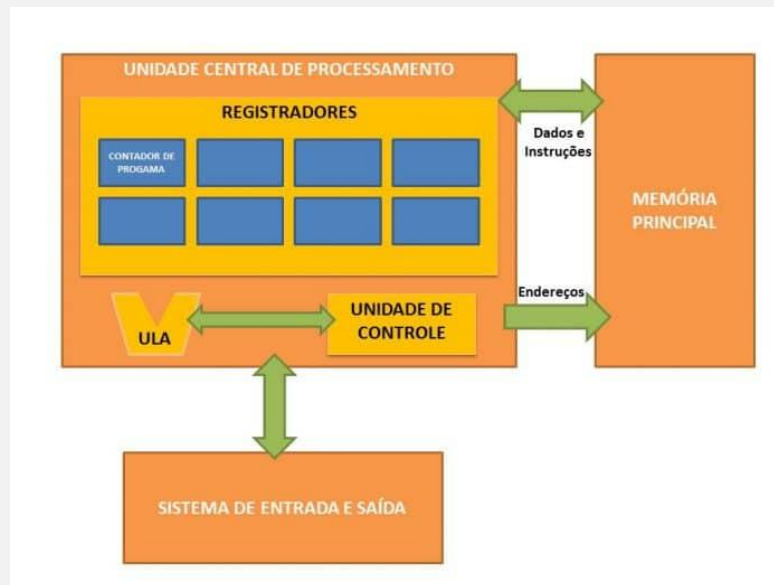
Componentes básicos de um computador

Embora tenham ocorrido revolucionárias transformações na área de Eletrônica, o microcomputador de hoje ainda mantém a mesma concepção funcional dos primeiros computadores eletrônicos. Tal concepção, conhecida como Arquitetura de Von Neumann, é definida da seguinte forma:

INFRAESTRUTURA COMPUTACIONAL

Componentes básicos de um computador

Uma unidade central de processamento recebe informações através de uma unidade de entrada de dados, processa estas informações segundo as especificações de um programa armazenado em uma unidade de memória, e devolve os resultados através de uma unidade de saída de dados.



► ARQUITETURA DE VON NEUMANN



INFRAESTRUTURA COMPUTACIONAL

O futuro

Os computadores quânticos também estão sendo largamente pesquisados ao redor do mundo, havendo iniciativas inclusive no Brasil. Nesse tipo de computador, são os átomos que desempenham o papel dos transistores. Ao contrário dos clássicos *bits* digitais (0 e 1), as menores unidades de informação de um computador quântico podem assumir qualquer valor entre zero e um. Dessa forma, existem previsões bem otimistas de que essa nova tecnologia substitua o silício (matéria-prima dos transistores) em pouco tempo.

INFRAESTRUTURA COMPUTACIONAL

Componentes básicos de um sistema computacional

B) Memória

Existem diversos tipos de memória em um computador (ex.: RAM (principal), ROM, *cache*, registradores), mas existe uma delas denominada memória principal, a qual é indispensável. A memória principal é tão importante quanto a UCP, pois sem ela não seria possível disponibilizar os programas e seus dados para o processamento pela CPU. Portanto, a memória é responsável por armazenar todos os programas que executam no computador e os dados que utilizam.

INFRAESTRUTURA COMPUTACIONAL

GERENCIAMENTO DE MEMÓRIA

Alocação Contínua Simples

MEMÓRIA



SISTEMA OPERACIONAL

PROCESSO

OCIOSO SE NÃO UTILIZADO

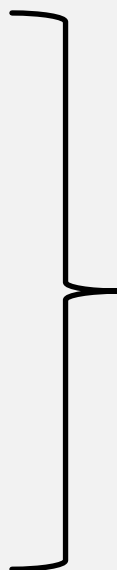
INFRAESTRUTURA COMPUTACIONAL

GERENCIAMENTO DE MEMÓRIA

Partição Fixa

MEMÓRIA

Sistema Operacional
1
2
3
4
5



Número limite de processos. Tirando o sistema operacional, o que sobra é dividido em N partes iguais.

Programa menor que uma das partes, desperdiça memória.

Programa maior que uma das partes, não roda.

GERENCIAMENTO DE MEMÓRIA

Partição Dinâmica

As partes são alocadas de acordo com o tamanho do processo.

=> Gera criação de tabelas para gerenciar áreas livres e ocupadas (aumento do trabalho de desenvolvimento do sistema operacional).

GERENCIAMENTO DE MEMÓRIA

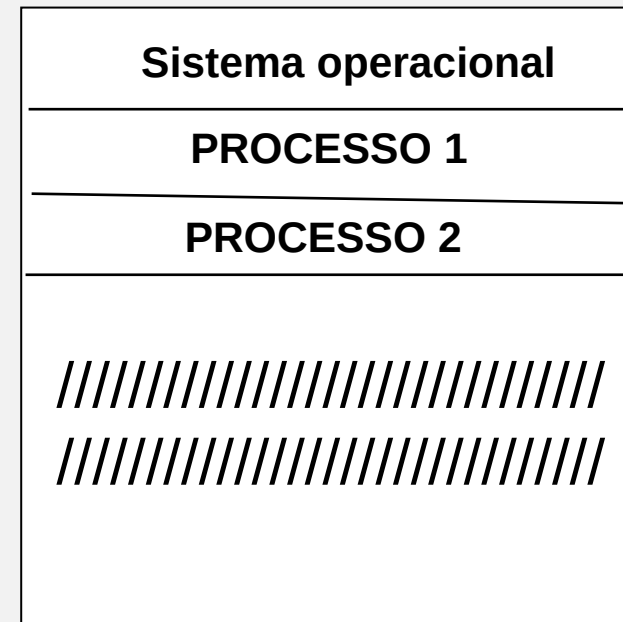
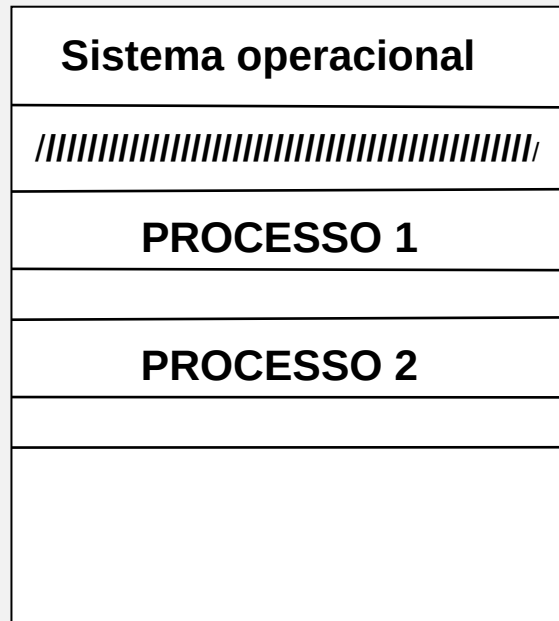
Partição Dinâmica

Desvantagens:

- Se o processo aguardando for maior que a partição livre, continua aguardando.
- Ociosidade (pode ocorrer processo aguardando embora exista memória livre).
- Fragmentação de memória (soma de espaços livres pequenos maior que a área utilizada).

GERENCIAMENTO DE MEMÓRIA

Partição com Relocação



GERENCIAMENTO DE MEMÓRIA

Partição com Relocação

- Mover um PROCESSO não garante que o PROCESSO execute corretamente na nova localização.
- As duas soluções mais comuns para o problema de relocação requerem suporte de hardware:
 - * Definir tipos de dados
 - * Usar registradores de relocação

INFRAESTRUTURA COMPUTACIONAL

GERENCIAMENTO DE MEMÓRIA

Paginação

Pág 0
Pág 1
Pág 2

PROCESSO

PAGINAÇÃO

0	2
1	4
2	7

MEMÓRIA

0
1000
2000
3000
4000
5000
6000
7000

BLOCO 2

BLOCO 4

BLOCO 7

24

GERENCIAMENTO DE MEMÓRIA

Paginação

- Não é necessário relocação pois o processo não necessita estar em áreas contínuas.
- As páginas possuem em geral o mesmo tamanho.
- Otimização: memória virtual = são carregadas na memória apenas as páginas em utilização, conforme forem necessárias. As não utilizadas ficam em disco.

GERENCIAMENTO DE MEMÓRIA

Paginação

Paginação por Demanda (Memória Virtual)

Quando um processo é colocado em execução, geralmente apenas uma página é carregada. As páginas subsequentes são carregadas por demanda.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Um numeral é um símbolo ou grupo de símbolos que representa um número em um determinado instante da evolução do homem. Tem-se que, numa determinada escrita ou época, os numerais diferenciaram-se dos números do mesmo modo que as palavras se diferenciaram das coisas a que se referem. Os símbolos "11", "onze" e "XI" (onze em latim) são numerais diferentes, representativos do mesmo número, apenas escrito em idiomas e épocas diferentes.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Um sistema de numeração (ou sistema numeral) é um sistema em que um conjunto de números são representados por numerais de uma forma consistente. Pode ser visto como o contexto que permite ao numeral "11" ser interpretado como o numeral romano para dois, o numeral binário para três ou o numeral decimal para onze.

SISTEMAS DE NUMERAÇÃO

Sistema Decimal

O sistema decimal é um sistema de numeração de posição que utiliza a base dez.

Símbolos da base Decimal: 0 1 2 3 4 5 6 7 8 9

Baseia-se em uma numeração de posição, onde os dez algarismos indo-arábicos : 0 1 2 3 4 5 6 7 8 9 servem para contar unidades, dezenas, centenas, etc. da direita para a esquerda.

Contrariamente à numeração romana, o algarismo árabe tem um valor diferente segundo sua posição no número: assim, em 111, o primeiro algarismo significa 100, o segundo algarismo 10 e o terceiro 1, enquanto que em VIII (oito em numeração romana) os três I significam todos 1.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Assim:

$$347 = 3 \cdot 100 + 4 \cdot 10 + 7 \cdot 1 = 3 \cdot 10^2 + 4 \cdot 10^1 + 7 \cdot 10^0$$

No sistema decimal o símbolo 0 (zero) posicionado à esquerda do número escrito não altera seu valor representativo.

Assim: 1; 01; 001 ou 0001 representam a mesma grandeza, neste caso a unidade. O símbolo zero posto à direita implica multiplicar a grandeza pela base, ou seja, por 10 (dez).

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Os sistemas de numeração mais utilizados na informática são:

- Base 2: também conhecido como sistema binário. Dígitos 0 e 1.**
- Base 8: o sistema octal também é um sistema posicional e foi utilizado na Informática como alternativa ao sistema binário. Dígitos de 0 a 7.**
- Base 16: o sistema hexadecimal é, talvez, um dos mais conhecidos da atualidade. Dígitos de 0 a F.**

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Vamos então conhecer os dígitos dessas bases:

INFRAESTRUTURA COMPUTACIONAL

DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2		2	2
3		3	3
4		4	4
5		5	5
6		6	6
7		7	7
8			8
9			9
10			
11			
12			
13			
14			
15			

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores decimais (dígitos de 0 a 9) em uma base binária (dígitos 0 e 1)?

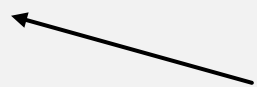
Para isso, precisamos realizar sucessivas divisões do número decimal por 2, pegando como resultado o resto da divisão e os quocientes, nessa ordem, da direita para a esquerda, de baixo para cima.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 2 em binário:

$$\begin{array}{r} \underline{2} \quad 2 \\ 0 \quad 1 \end{array}$$


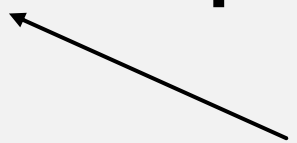
Então $2 = (10)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 3 em binário:

$$\begin{array}{r} 3 \quad 2 \\ \hline 1 \quad 1 \end{array}$$


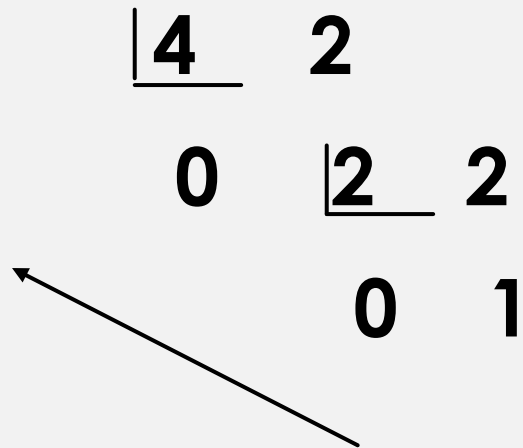
Então $3 = (11)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 4 em binário:



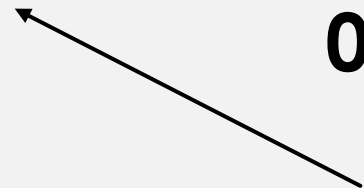
Então $4 = (100)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 5 em binário:

$$\begin{array}{r} \underline{5} \quad 2 \\ 1 \quad \underline{2} \quad 2 \\ \quad \quad 0 \quad 1 \end{array}$$


Então $5 = (101)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 15 em binário:

$$\begin{array}{r} \underline{15} \quad 2 \\ 1 \quad \underline{7} \quad 2 \\ \quad 1 \quad \underline{3} \quad 2 \\ \qquad \quad 1 \quad 1 \end{array}$$


Então $15 = (1111)_2$

Vamos então completar a tabela.

INFRAESTRUTURA COMPUTACIONAL

DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6		6	6
7		7	7
8			8
9			9
10			
11			
12			
13			
14			
15	1111		

ORGANIZAÇÃO DE COMPUTADORES

DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000		8
9	1001		9
10	1010		
11	1011		
12	1100		
13	1101		
14	1110		
15	1111		

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores decimais (dígitos de 0 a 9) em uma base octal (dígitos 0 a 7)?

Para isso, precisamos realizar sucessivas divisões do número decimal por 8, pegando como resultado o resto da divisão e os quocientes, nessa ordem, da direita para a esquerda, de baixo para cima.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 8 em octal:

$$\begin{array}{r} 8 \quad 8 \\ \hline 0 \quad 1 \end{array}$$


Então $8 = (10)_8$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 9 em octal:

$$\begin{array}{r} 9 \overline{) 8} \\ 1 \\ \hline 1 \end{array}$$

←

Então $9 = (11)_8$

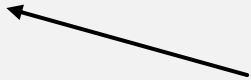
INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 10 em octal:

10	8
2	1



Então $10 = (12)_8$

INFRAESTRUTURA COMPUTACIONAL

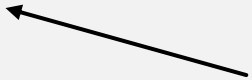
SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 11 em octal:

11 8

3 1



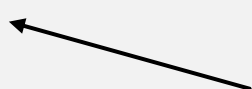
Então $11 = (13)_8$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 15 em octal:

$$\begin{array}{r} 15 \div 8 \\ 7 \quad 1 \end{array}$$


Então $15 = (17)_8$

Vamos então completar a tabela.

INFRAESTRUTURA COMPUTACIONAL

DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	
11	1011	13	
12	1100		
13	1101		
14	1110		
15	1111	17	

ORGANIZAÇÃO DE COMPUTADORES

DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	
11	1011	13	
12	1100	14	
13	1101	15	
14	1110	16	
15	1111	17	

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores decimais (dígitos de 0 a 9) em uma base hexadecimal (dígitos 0 a F)?

Para isso, precisamos realizar sucessivas divisões do número decimal por 16, pegando como resultado o resto da divisão e os quocientes, nessa ordem, da direita para a esquerda, de baixo para cima.

Vamos então completar a tabela.

INFRAESTRUTURA COMPUTACIONAL

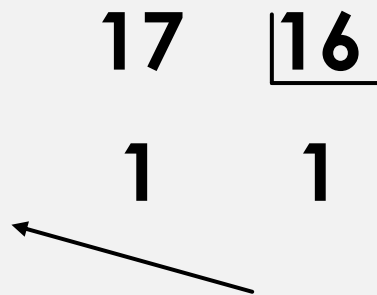
DECIMAL	BINÁRIO	OCTAL	HEXADECIMAL
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 17 em hexadecimal:

$$\begin{array}{r|l} 17 & 16 \\ \hline 1 & 1 \end{array}$$


Então $17 = (11)_{16}$ ou H

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 30 em hexadecimal:

$$\begin{array}{r|l} 30 & 16 \\ \hline 14 & 1 \end{array}$$

←

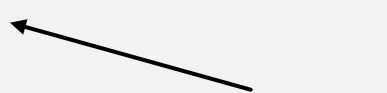
Então $30 = (1E)_{16}$ ou H

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o decimal 65 em hexadecimal:

$$\begin{array}{r|l} 65 & 16 \\ \hline 1 & 4 \end{array}$$


Então $65 = (41)_{16}$ ou H

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

1) Transformar os números abaixo de decimal para binário

a) $32 = (\quad)_2$

b) $45 = (\quad)_2$

c) $56 = (\quad)_2$

d) $18 = (\quad)_2$

e) $27 = (\quad)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS (GABARITO)

1) Transformar os números abaixo de decimal para binário

A) $32 = (100000)_2$

B) $45 = (101101)_2$

C) $56 = (111000)_2$

D) $18 = (10010)_2$

E) $27 = (11011)_2$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

2) Transformar os números abaixo de decimal para octal

a) $32 = (\quad)_8$

b) $45 = (\quad)_8$

c) $56 = (\quad)_8$

d) $18 = (\quad)_8$

e) $27 = (\quad)_8$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS - GABARITO

2) Transformar os números abaixo de decimal para octal

A) $32 = (40)_8$

B) $45 = (55)_8$

C) $56 = (70)_8$

D) $18 = (22)_8$

E) $27 = (33)_8$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

1) Transformar os números abaixo de decimal para hexadecimal

A) $32 = (\quad)_{16}$

B) $45 = (\quad)_H$

C) $56 = (\quad)_H$

D) $18 = (\quad)_{16}$

E) $27 = (\quad)_{16}$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS - GABARITO

3) Transformar os números abaixo de decimal para hexadecimal

A) $32 = (20)_{16}$

B) $45 = (2D)_H$

C) $56 = (38)_H$

D) $18 = (12)_{16}$

E) $27 = (1B)_{16}$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores binários (dígitos 0 e 1) em uma base decimal (dígitos de 0 a 9)?

Para isso, precisamos obter o produto de cada dígito binário, tomados da direita para a esquerda, pelas potências sucessivas de 2, a partir de 0.

O resultado será a soma dos produtos.

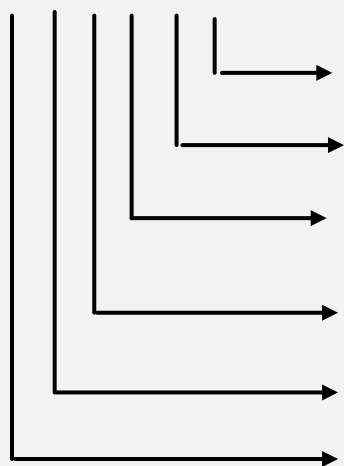
INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o número $(101011)_2$ em decimal.

101011



$$1 \times 2^0 = 1$$

$$1 \times 2^1 = 2$$

$$0 \times 2^2 = 0$$

$$1 \times 2^3 = 8$$

$$0 \times 2^4 = 0$$

$$1 \times 2^5 = 32$$

$$\text{Então, } (101011)_2 = 43$$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores na base octal (dígitos de 0 e 7) em uma base decimal (dígitos de 0 a 9)?

Para isso, precisamos obter o produto de cada dígito octal, tomados da direita para a esquerda, pelas potências sucessivas de 8, a partir de 0.

O resultado será a soma dos produtos.

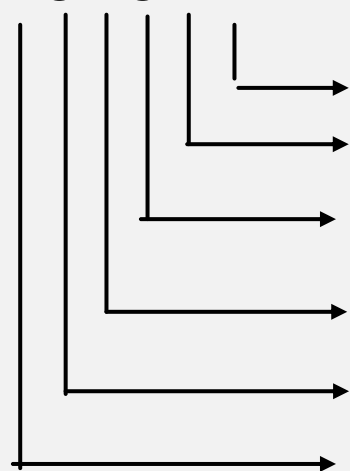
INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o número $(101011)_8$ em decimal.

101011



$$1 \times 8^0 = 1$$

$$0 \times 8^1 = 0$$

$$1 \times 8^2 = 64$$

$$0 \times 8^3 = 0$$

$$1 \times 8^4 = 4096$$

$$1 \times 8^5 = 32768$$

$$\text{Então, } (101011)_8 = 33289$$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Como representar valores na base hexadecimal (dígitos de 0 a F) em uma base decimal (dígitos de 0 a 9)?

Para isso, precisamos obter o produto de cada dígito hexadecimal, tomados da direita para a esquerda, pelas potências sucessivas de 16, a partir de 0.

O resultado será a soma dos produtos.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO

Exemplos:

Transformar o número $(1F1A)_{16}$ em decimal.

1F1A

1	F	1	A
→	→	→	→
10	15	1	10
$\times 16^0 =$	$\times 16^1 =$	$\times 16^2 =$	$\times 16^3 =$
10	16	3840	65536

Então, $(1F1A)_{16} = 69402$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

1) Transformar os números abaixo de binário para decimal

A) $(10110)_2 =$

B) $(11100)_2 =$

C) $(101010)_2 =$

D) $(110011)_2 =$

E) $(100000)_2 =$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS (GABARITO)

1) Transformar os números abaixo de binário para decimal

A) $(10110)_2 = 22$

B) $(11100)_2 = 28$

C) $(101010)_2 = 42$

D) $(110011)_2 = 51$

E) $(100000)_2 = 32$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

1) Transformar os números abaixo de octal para decimal

A) $(123)_8 =$

B) $(37)_8 =$

C) $(1033)_8 =$

D) $(2001)_8 =$

E) $(378)_8 =$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS (GABARITO)

1) Transformar os números abaixo de octal para decimal

a) $(123)_8 = 83$

b) $(37)_8 = 31$

c) $(1033)_8 = 539$

d) $(2001)_8 = 1025$

e) $(378)_8 = 256$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO - EXERCÍCIOS

1) Transformar os números abaixo de hexadecimal para decimal

a) $(123)_{16} =$

b) $(AB)_{16} =$

c) $(FOFA)_H =$

d) $(1002)_H =$

e) $(B0A)_{16} =$

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXERCÍCIOS (GABARITO)

1) Transformar os números abaixo de hexadecimal para decimal

a) $(123)_{16} = 291$

b) $(AB)_{16} = 171$

c) $(FOFA)_H = 61690$

d) $(1002)_H = 4098$

e) $(B0A)_{16} = 2826$

INFRAESTRUTURA COMPUTACIONAL

EXERCÍCIOS DE FIXAÇÃO

Converta o número apresentado em uma base para todas as outras bases estudadas:

a) $(101011011)_2 \Rightarrow (?)_8 \quad (?)_{10} \quad (?)_{16}$

b) $(17256)_8 \Rightarrow (?)_2 \quad (?)_{10} \quad (?)_{16}$

c) $(21518)_{10} \Rightarrow (?)_2 \quad (?)_8 \quad (?)_{16}$

d) $(DAD0)_{16} \Rightarrow (?)_2 \quad (?)_8 \quad (?)_{10}$



INFRAESTRUTURA COMPUTACIONAL

CÓDIGOS BINÁRIOS PARA O ALFABETO

CÓDIGO BINARIO

A	01000001
B	01000010
C	01000011
D	01000100
E	01000101
F	01000110
G	01000111
H	01001000
I	01001001
J	01001010
K	01001011
L	01001100
M	01001101

N	01001110
O	01001111
P	01010000
Q	01010001
R	01010010
S	01010011
T	01010100
U	01010101
V	01010110
W	01010111
X	01011000
Y	01011001
Z	01011010

a	01100001
b	01100010
c	01100011
d	01100100
e	01100101
f	01100110
g	01100111
h	01101000
i	01101001
j	01101010
k	01101011
l	01101100
m	01101101

n	01101110
o	01101111
p	01110000
q	01110001
r	01110010
s	01110011
t	01110100
u	01110101
v	01110110
w	01110111
x	01111000
y	01111001
z	01111010

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para a composição de números IP (Internet Protocol)

Os números IP identificam uma conexão de rede e são compostos por 4 octetos.

São chamados de octetos por serem equivalentes a 2^8 .

O maior número IP teórico seria 255.255.255.255

A maior quantidade de números IP's válidos seria obtida por:

$256 * 256 * 256 * 256 = 4.294.967.296$ IP's válidos

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para a composição de números IP (Internet Protocol)

$$256 * 256 * 256 * 256 = 4.294.967.296 \text{ IP's válidos}$$

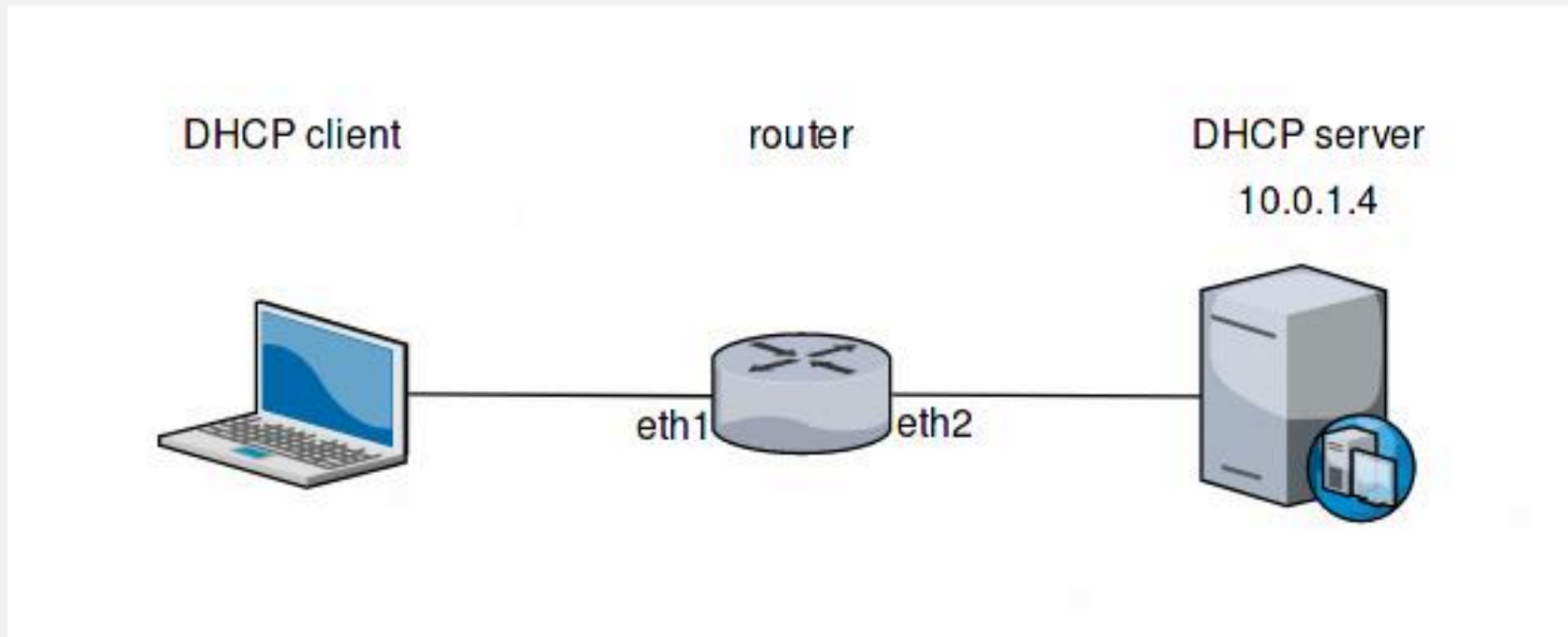
Esses números seriam suficientes para identificar todos os computadores do mundo conectados na Internet? Não!

Por isso são criados, a partir de números IP válidos, números IP dinâmicos, para identificar internamente em uma rede os computadores.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para a composição de números IP (Internet Protocol)
DHCP (Dynamic Host Configuration Protocol)



INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

Em sistemas operacionais modernos são implementados três níveis de acesso, os quais são subdivididos em três maneiras de operação.

Níveis: dono, grupo, universo.

Maneiras: ler, escrever, executar.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

Assim sendo, um arquivo ou pasta teria as seguintes possibilidades de restrições de acesso:

 R W X R W X R W X nome.ext

└──┬──┘ └──┬──┘ └──┬──┘

DONO GRUPO UNIVERSO

R = Read; W = Write; X = Execute

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

Se substituíssemos as letras por dígitos que representem a permissão ou a negação de acesso em cada nível teríamos, por exemplo:

— 1 1 1 1 1 1 1 1 1 nome.ext
 └──┬──┘ └──┬──┘ └──┬──┘
 DONO GRUPO UNIVERSO

Ou seja: todos os níveis liberados para acessar de todas as maneiras.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

E se eu quisesse implementar as seguintes permissões:

- Dono: R W X - Grupo: R W _ - Universo: _ _ _

_	R	W	X		R	W	_	_	_	_	nome.ext
	└───┘				└───┘			└───┘			
	DONO				GRUPO			UNIVERSO			

Ou:

_	1	1	1		1	1	0		0	0	0	nome.ext
---	---	---	---	--	---	---	---	--	---	---	---	----------

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

Se substituísimos as permissões por valores decimais equivalentes aos binários formados, teremos:

— 1 1 1 1 1 0 0 0 0 nome.ext
 └──┬──┘ └──┬──┘ └──┬──┘
 7 6 0

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

Por exemplo em UNIX (qualquer das versões) o comando CHMOD permite alterar as permissões de acesso a um arquivo ou pasta, da seguinte maneira:

CHMOD 777 nome.ext

Resultado: _ R W X R W X R W X nome .ext

CHMOD 7 0 0 nome.ext

Resultado: _ R W X _ _ _ _ _ nome.ext

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

CHMOD 777 nome.ext

Resultado: _ R W X R W X R W X nome .ext

Nesse caso, todas as permissões de acesso foram dadas a todos os usuários.

CHMOD 7 0 0 nome.ext

Resultado: _ R W X _ _ _ _ _ nome.ext

Nesse caso, foram dadas todas as permissões de acesso ao dono e nenhuma permissão aos demais.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para implementar sistemas de segurança em arquivos/pastas.

CHMOD 761 nome.ext

Resultado: _ R W X R W _ _ _ X nome .ext

1	1	1	1	1	0	0	0	1
└──┬──┘			└──┬──┘			└──┬──┘		
7			6			1		

Nesse caso, todas as permissões foram dadas ao dono, somente leitura e escrita para o grupo e somente execução para o universo.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Exercícios

Apresente as permissões para o arquivo teste.txt, conforme definido pela instrução CHMOD:

- a) CHMOD 732
- b) CHMOD 100
- c) CHMOD 521

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Exercícios - Gabarito

Apresente as permissões para o arquivo teste.txt, conforme definido pela instrução CHMOD:

a) CHMOD 732

_ R W X _ W X _ W _ teste.txt

b) CHMOD 100

_ _ _ 1 _ _ _ _ _ teste.txt

c) CHMOD 521

_ R _ X _ W _ _ _ 1 teste.txt

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Qual é o equivalente ao CHMOD no Windows?

Não pode haver um equivalente exato, pois o Unix e o DOS / Windows usam atributos para propósitos diferentes e o modelo de segurança é diferente:

Nos sistemas de arquivos do Windows, existem atributos "ocultos" (H) e "sistema" (S) que não têm equivalente no Unix; lá, os arquivos são ocultados acrescentando o nome com um ponto (.).

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Qual é o equivalente ao CHMOD no Windows?

Também não há equivalente ao atributo "archive" (A) do Windows.

Não há equivalente aos atributos "executáveis" (x) do Unix nos atributos de arquivo do DOS / Windows.

Existe um equivalente ao atributo "diretório" (D) do Windows (mas não pode ser alterado de qualquer maneira).

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Qual é o equivalente ao CHMOD no Windows?

Nos sistemas de arquivos Unix, cada entrada pertence a exatamente um usuário e exatamente um grupo, e a leitura / gravação / execução pode ser permitida para cada um deles e para outros.

As ACLs (como usadas pelo Windows) são ainda mais flexíveis, mas também mais complicadas, e a sintaxe da linha de comando é complexa.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

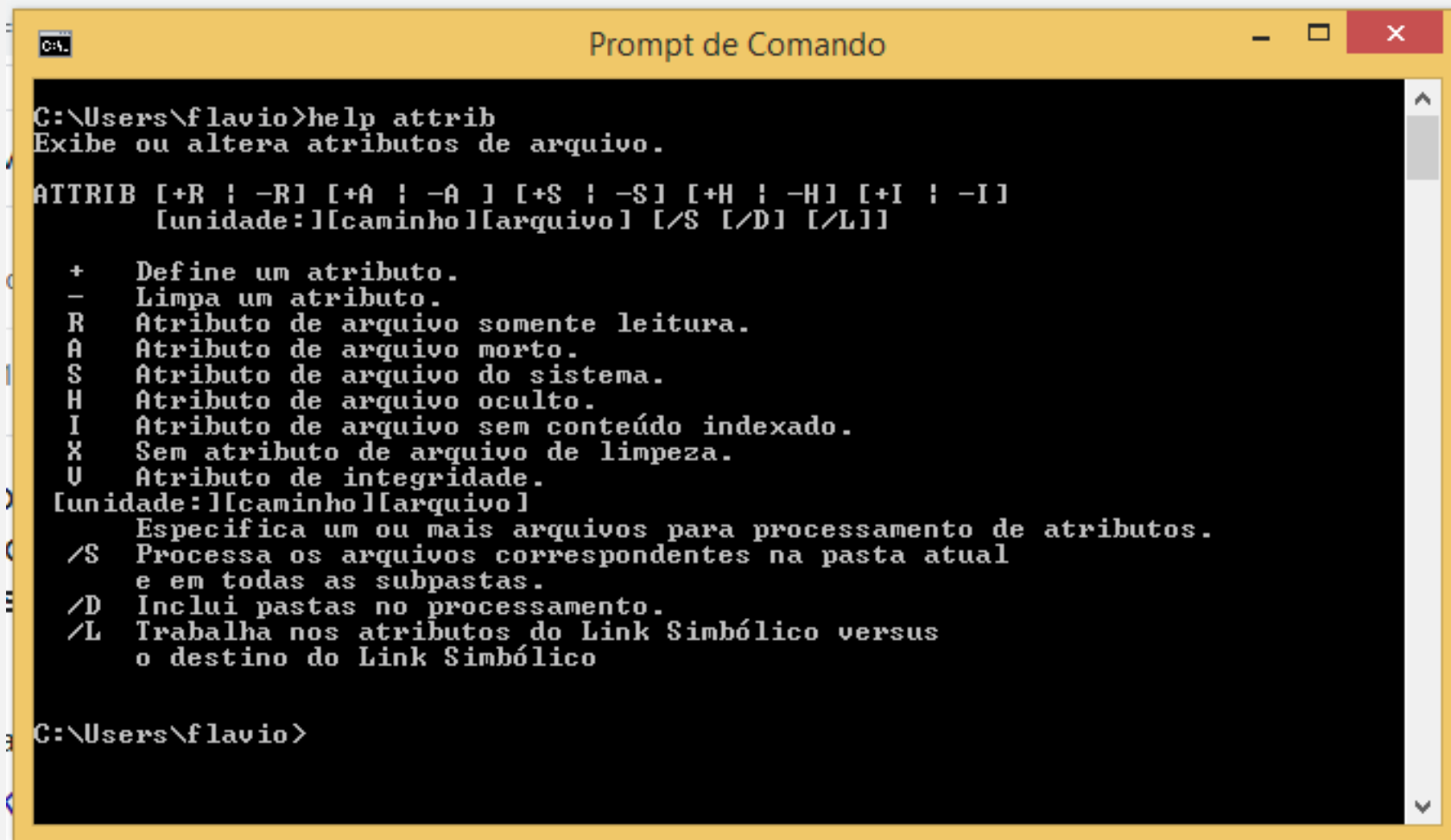
Qual é o equivalente ao CHMOD no Windows?

O atributo de arquivo DOS R (somente leitura) é o que pode ser considerado como tendo um equivalente: esse *conjunto de atributos* é aproximadamente o atributo W para todos os que estão faltando; mas a permissão para alterar esse atributo está sujeita às ACLs.

INFRAESTRUTURA COMPUTACIONAL

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Qual é o equivalente ao CHMOD no Windows?



```
C:\Users\flavio>help attrib
Exibe ou altera atributos de arquivo.

ATTRIB [+R | -R] [+A | -A] [+S | -S] [+H | -H] [+I | -I]
        [unidade:][caminho][arquivo] [/S [/D] [/L]]

+   Define um atributo.
-   Limpa um atributo.
R   Atributo de arquivo somente leitura.
A   Atributo de arquivo morto.
S   Atributo de arquivo do sistema.
H   Atributo de arquivo oculto.
I   Atributo de arquivo sem conteúdo indexado.
X   Sem atributo de arquivo de limpeza.
U   Atributo de integridade.
[unidade:][caminho][arquivo]
    Especifica um ou mais arquivos para processamento de atributos.
/S   Processa os arquivos correspondentes na pasta atual
     e em todas as subpastas.
/D   Inclui pastas no processamento.
/L   Trabalha nos atributos do Link Simbólico versus
     o destino do Link Simbólico

C:\Users\flavio>
```

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Na documentação MDN (comunidade de desenvolvedores) encontra-se uma lista enorme de nomes de cores que podem ser usadas no CSS com seus respectivos valores em hexadecimal. Por exemplo:

Em Inglês	Em Português	Em Hexa
Black	Preto	#000000
Silver	Prata	#c0c0c0
White	Branco	#ffffff
Red	Vermelho	#ff0000

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Para aplicar um fundo vermelho (red) a uma div pode-se usar:

Usando o nome da cor	Usando valor em hexa
<pre>div { background-color: red; }</pre>	<pre>div { background-color: #ff0000;</pre>

Ou usando RGB

```
div {  
background-color: rgb (255, 0, 0);  
}
```

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Nos anos 60, o código ASCII (American Standard Code for Information Interchange) foi adotado como código padrão. Ele permite a codificação de caracteres em 8 bits, dando a possibilidade de formação de 256 caracteres.

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

A memória do computador conserva todos os dados sob a forma digital. Não existe um método para armazenar diretamente os caracteres. Cada caractere possui o seu equivalente em código numérico, é o ASCII.

O código ASCII básico representava os caracteres em 7 bits (ou seja, 128 caracteres possíveis, de 0 a 127).

Os códigos 0 a 31 não são caracteres; são chamados de caracteres de controle porque permitem fazer ações como carriage return (CR - retorno do carro em português) e bip sonoro (BEL). Os códigos 65 a 90 representam as maiúsculas e os códigos 97 a 122 representam as minúsculas. Basta alterar o 6º bit para passar das maiúsculas para as minúsculas, isto é, acrescentar 32 ao código ASCII em base decimal.

SISTEMAS DE NUMERAÇÃO – EXEMPLOS DE APLICAÇÃO

Tabela ASC II (Exemplos)

Caractere	ASC II	Hexa
LF (Line Feed, salto de linha)	10	0A
CR (Carriage return, retorno do carro)	13	0D
DC1 (Device control 1)	17	11
ESC (Escape, caractere ESC)	27	1B
%	37	25

Sistemas de numeração – Exemplos de aplicação

PIXEL

A palavra *pixel* é oriunda da junção dos termos *picture* e *element*, formando, ao pé da letra, a expressão elemento de imagem. Ao visualizarmos uma imagem com alto índice de aproximação, é possível identificar pequenos quadrados coloridos nela, que, somados, formam o desenho completo.

Sistemas de numeração – Exemplos de aplicação

PIXEL

Esses pontos, que são a menor parte de uma imagem, levam o nome de *pixels*. A partir da noção do *pixel* como uma medida da qualidade das imagens, foi propagado o termo “resolução” para atribuir quantos *pixels* em altura e largura uma foto tem.

Sistemas de numeração – Exemplos de aplicação

PIXEL

Um dispositivo com 1 *Megapixel* de resolução é capaz de ampliar imagens com cerca de um milhão de *pixels* em tamanho.

Uma câmera de 1,3 *Megapixels*, por exemplo, é capaz de gerar 1.300.000 *pixels* dentro de uma única imagem, o que, em termos de resolução, equivale a uma foto com 1280 *pixels* de largura por 1024 de altura, somando um total de 1.310.720 pontos.

Sistemas de numeração – Exemplos de aplicação

PIXEL

Em termos práticos, é perceptível identificar a mudança na qualidade das texturas ao modificarmos a resolução do nosso monitor, em especial no que se refere a jogos. Quanto maior a deixamos, mais bem delineadas as imagens e elementos do Windows se tornam.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens

Sabe-se que com as três cores primárias, vermelho, verde e azul, pode-se criar as outras cores do espectro. No caso de imagens *true colors*, cada pixel é representado por 24 bits, 8 bits para cada cor. Com esses 8 bits, pode-se definir 256 tons para cada uma das três cores primárias. Combinando-se essas três cores, pode-se ter até 16.777.216 diferentes tons. O preço a se pagar por essa definição é a quantidade de bytes necessária para armazenar uma imagem.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens

Se pegarmos, por exemplo, uma imagem *true color* VGA de 640×480 pixels, precisa-se de:

$$640 \times 480 \times 24 = 7.372.800 \text{ pixels}$$

ou 921,6 Kbytes ($7372800 / 8$) para ser representada.

Uma imagem full HD 1080×1920 precisaria de 6.22 Mbytes para ser representada.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens

Já uma imagem binária é uma imagem onde cada pixel é representado apenas por um bit, ou seja, um pixel pode ser ou preto ou branco. Pode-se notar facilmente que uma imagem binária exige 24 vezes menos bits para ser representada. Neste caso uma imagem VGA binária $(480 \times 640)/8$, precisa de apenas 38,4 Kbytes para ser representada. E uma imagem full HD apenas 259,2 Kbytes $(1080 \times 1920)/8$.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens

Para se criar uma imagem binária, primeiro é preciso transformar uma imagem colorida em uma imagem em tons de cinza. Uma imagem em tons de cinza é uma imagem onde cada pixel é representado pela intensidade da luz apenas. Normalmente usa-se 8 bits para se ter até 256 tons de cinza.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens

A transformação é dada pela seguinte fórmula:

$$y = 0,2126 * R + 0,7152 * G + 0,0722 * B$$

onde y é o valor da intensidade, R é o nível de vermelho, G é o nível de verde e B o nível de azul.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens



Foto em tons de cinza.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

Algoritmo 1: Threshold (Limiar)

Este é o método mais simples e intuitivo. Para cada pixel, verifica-se se ele possui um valor acima ou abaixo de um limite (threshold). Estando acima, define-se esse pixel com o valor 1, e estando abaixo, define-se o valor como 0.

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

Algoritmo 1: Threshold



Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

Algoritmo 2: Ordered dithering (Pontilhamento ordenado)

Neste método usa-se uma matriz para criar um erro aleatório na quantização dos pixels. Normalmente usa-se matrizes de *Bayer*, porém pode-se usar qualquer outra matriz. A vantagem desse algoritmo é a velocidade pois trata-se a imagem em blocos de pixels, definido pelo tamanho da matriz. Na imagem de exemplo, usamos uma matriz de Bayer 8×8.

- ▶ **Sistemas de numeração – Exemplos de aplicação**
- ▶ **PIXEL – Resolução de imagens – Binarização de imagens Algoritmo 2: Ordered dithering**



Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

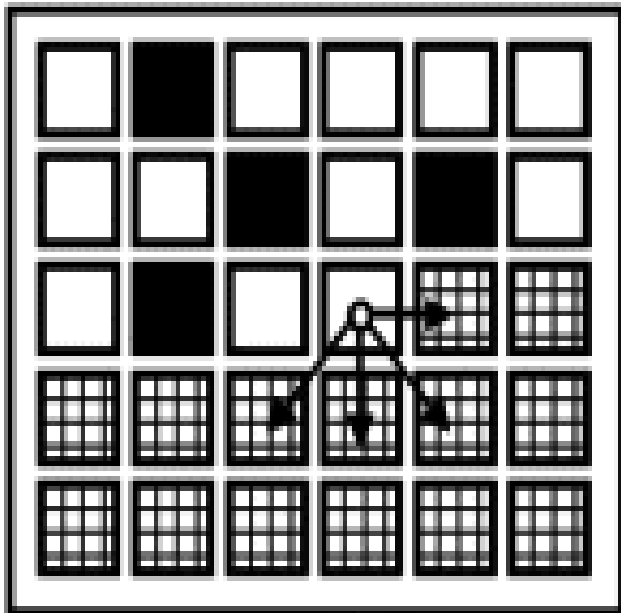
Algoritmo 3: Error diffusion (Difusão de erro)

No exemplo foi utilizado o algoritmo de *Floyd–Steinberg*, o qual tenta difundir o erro do pixel com os 4 pixels vizinhos. A difusão de erro do pixel é dividida nas seguintes proporções:

Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

Algoritmo 3: Error diffusion



direita: 7/16

baixo-esquerda: 3/16

baixo: 5/16

baixo-direita: 1/16

Nota-se uma melhora na qualidade da imagem comparada às técnicas anteriores ao custo de processamento maior.

- ▶ Sistemas de numeração – Exemplos de aplicação
- ▶ PIXEL – Resolução de imagens – Binarização de imagens Algoritmo 3: Error diffusion



Sistemas de numeração – Exemplos de aplicação

PIXEL – Resolução de imagens – Binarização de imagens

Comparando resultados



ORIGINAL



THRESHOLD



ORDERED
DITHERING



ERROR
DIFUSION

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

Um computador funciona por meio da execução sistemática de instruções que o orientam a realizar algum tipo de operação sobre valores (numéricos, alfanuméricos ou lógicos). Esses valores são genericamente conhecidos como dados.

Os dados são convertidos internamente em um código de armazenamento no formato binário.

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

Qualquer que tenha sido a linguagem de programação utilizada para escrever o programa, ela deverá ser convertida para código-objeto (código binário) e, em seguida, para o código executável (conjunto de códigos de máquina), o qual é gerado pelo compilador da linguagem).

Tipos de dados

Definem para o sistema como cada dado deverá ser manipulado, pois conforme citado anteriormente, cada tipo de dado recebe um tratamento diferenciado pelo processador.

Exemplo:

```
VAR X:=INTEGER;
```

```
VAR X:=REAL;
```

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

Tipos de dados

Os termos INTEGER e REAL são interpretados de modo diferente, acarretando alterações significativas tanto no modo de organizar os bits que representam um número quanto na sequência de etapas do algoritmo de execução de uma operação aritmética com o número.

De modo geral, as seguintes formas de dados são mais utilizadas nos programas atuais de computadores (formas primitivas, entendidas pelo hardware).

REPRESENTAÇÃO DE DADOS

Tipos de dados

- Tipo Caractere: dados sob forma de caractere;
- Tipo Lógico: dados sob forma lógica;
- Tipo Numérico: dados sob forma numérica.

Outras formas mais complexas são permitidas em certas linguagens modernas (como tipo REGISTRO, tipo ARRAY, tipo INDEX, tipo POINTER etc.). No entanto, durante o processo de compilação, os dados acabam sendo convertidos finalmente nas formas primitivas já mencionadas, para que o hardware possa executá-las.

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

– Tipo Caractere

A representação interna de informações em um computador é realizada através de uma correspondência entre o símbolo da informação e o grupo de algarismos binários (bits). Cada símbolo (caractere, número ou símbolo) possui uma identificação específica.

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

– Tipo Caractere

BCD – Binary Code Decimal	Utiliza 6 bits/caracteres, codificando 64 caracteres.
EBCDIC – Extended Binary Coded Decimal Interchange Code	Exclusivo da IBM, utilizando 8 bits para codificar 256 caracteres.
ASCII – American Standart Code for Information Interchange	ASCII – usado pelos demais fabricantes. Utiliza oito bits/caractere em sua versão extendida, codificando 256 caracteres
UNICODE	Código que utiliza 16 bits/símbolo, podendo representar 65.536 símbolos diferentes. Pretende codificar em um único código os símbolos de todas as linguagens conhecidas no mundo. Está sendo desenvolvido por um consórcio desde 1991 (www.unicode.org).

INFRAESTRUTURA COMPUTACIONAL

REPRESENTAÇÃO DE DADOS

– Tipo Lógico

Permite a utilização de variáveis que possuem apenas dois valores para representação: Falso (bit 0) e Verdadeiro (bit 1).

Porta lógica	Definição
AND	O operador lógico AND é definido de modo que o resultado da operação com ele será VERDADE se e somente se todas as variáveis de entrada forem VERDADE (=1). Caso contrário, o resultado será FALSO (=0)
OR	O resultado da operação será VERDADE (=1) se um operando (ou variável lógica) ou o outro for verdadeiro. Basta que apenas um dos operandos seja verdadeiro. Caso contrário, o resultado será FALSO (=0). Operadores lógicos OR também são largamente utilizados em lógica digital ou na definição de condições em comandos de decisão de certas linguagens de programação.
NOT	É definido de modo a produzir na saída um resultado de valor oposto (ou inverso) ao da variável de entrada. É usado apenas com uma única variável. Desse modo, se a variável tem o valor 0 (FALSO), o resultado da operação NOT sobre essa variável será 1 (VERDADE), e se a variável for igual a 1 (VERDADE), então o resultado do NOT será 0 (FALSO).
XOR	O operador lógico XOR (EXCLUSIVE-OR) ou OU EXCLUSIVO é definido de modo a prover um resultado VERDADEIRO se apenas uma das variáveis ou operadores for VERDADEIRA. Sendo $X = A \text{ XOR } B$, o resultado X será VERDADE se exclusivamente (daí o nome OU EXCLUSIVO) A OU B for VERDADE. Caso ambos sejam “VERDADE” ou ambos “FALSO”, então o resultado será FALSO.
NAND	Negação do operador lógico AND
NOR	Negação do operador lógico OR.

Representação de dados

– Tipo Numérico

Como os computadores são elementos binários, a forma mais eficiente de representar números deve ser “binária”, isto é, converter o número diretamente de decimal para seu correspondente valor binário. Deste modo a ULA poderá executar as operações mais rapidamente.

A quantidade de dígitos disponíveis no sistema de computação (processador) para representar números é um problema relevante. Na matemática os números reais existentes são infinitos; no entanto, computadores são máquinas finitas onde as células e registradores possuem tamanho finito mas que têm capacidade de representar uma quantidade infinita de números. Desse modo surge o aspecto denominado overflow ou estouro da capacidade de representar números. Há também o underflow, que se caracteriza por ocorrer um resultado cujo valor é menor que o menor valor representável.

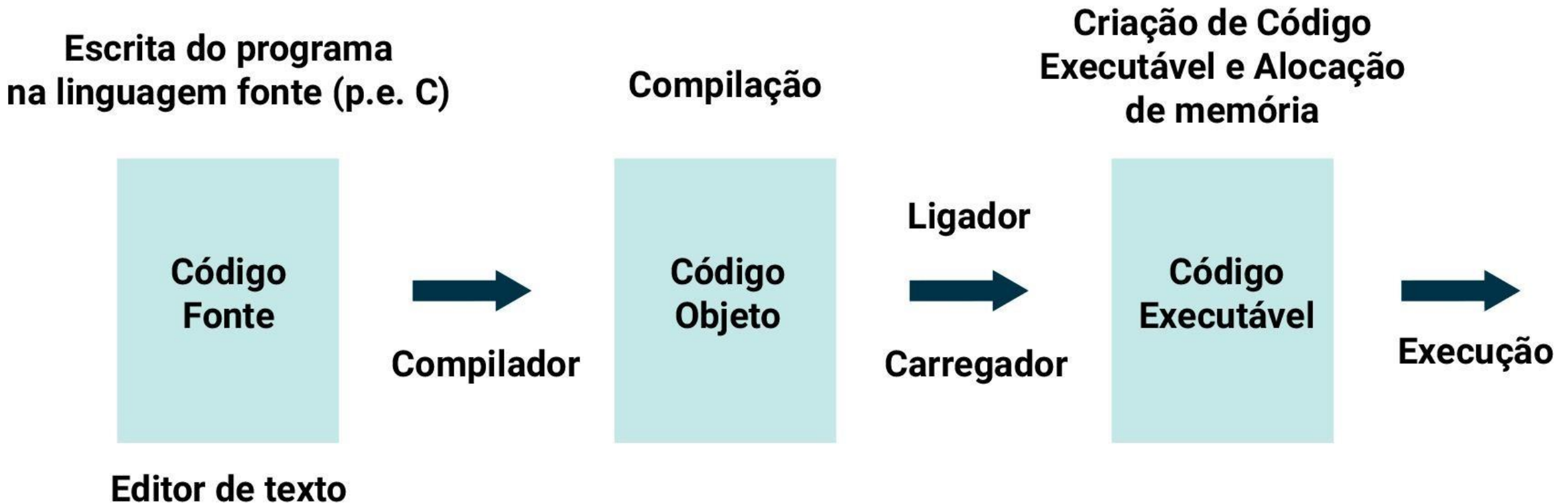
Tempo de compilação X Tempo de execução

O ***compile time*** ou tempo de compilação é tudo o que ocorre durante o processo de compilação, tudo o que pode ser detectado, gerado, otimizado, realizado quando o código está sendo compilado. Em geral é onde se pega erros sintáticos, léxicos e semânticos, de tipagem.

O compilador é um programa capaz de conseguir traduzir o código-fonte de um software, escrito em uma linguagem de alto nível, para uma espécie de programa equivalente, escrito em outra linguagem (de baixo nível), que seja semanticamente equivalente, mas capaz de ser lida pelos processadores.

INFRAESTRUTURA COMPUTACIONAL

Tempo de compilação X Tempo de execução



Tempo de compilação X Tempo de execução

O *run time* ou tempo de execução é tudo o que ocorre quando o código já está executando. Então ali, se houver um problema, pode ser tarde demais para solucionar. Por outro lado pode ser que só naquele momento com o dado certo seja possível entender o que ocorre, o que pode ser otimizado etc.

INFRAESTRUTURA COMPUTACIONAL

Tempo de compilação X Tempo de execução



Tempo de compilação X Tempo de execução

O que pode dar errado no momento da compilação:

- Erros de sintaxe
- Erros de digitação
- (Raramente) falhas de compilador

Tempo de compilação X Tempo de execução

Se a compilação tiver êxito:

- O programa foi bem formado - um programa significativo em qualquer idioma.
- É possível começar a executar o programa. (O programa pode falhar imediatamente, mas pelo menos podemos tentar)

Tempo de compilação X Tempo de execução

As entradas e saídas da compilação:

- A entrada era o programa que estava sendo compilado, além de quaisquer arquivos de cabeçalho, interfaces, bibliotecas ou outro recurso que ele precisava *importar* para ser compilado.
- Esperamos que a saída seja código de montagem ou código de objeto realocável ou mesmo um programa executável. Ou se algo der errado, a saída é um monte de mensagens de erro.

Tempo de compilação X Tempo de execução

O que pode dar errado em tempo de execução:

- Divisão por zero
- Referência a um ponteiro nulo
- Falta de memória

Também pode haver erros que são detectados pelo próprio programa:

- Tentando abrir um arquivo que não existe.
- Tentando encontrar uma página da web e descobrindo que um suposto URL não está bem formado.

Tempo de compilação X Tempo de execução

Se o tempo de execução for bem-sucedido, o programa termina (ou continua) sem travar.

Entradas e saídas são inteiramente da responsabilidade do programador. Arquivos, janelas na tela, pacotes de rede, trabalhos enviados para a impressora, o nome dele etc.

Se o programa lança mísseis, isso é uma saída e acontece apenas em tempo de execução.

Tempo de compilação X Tempo de execução

Exemplo:

Suponhamos que desejamos calcular a média entre dois números, que serão digitados, e o resultado será apresentado na tela.

Para isso utilizaremos a sintaxe da linguagem Pascal.

Tempo de compilação X Tempo de execução

Exemplo 1:

Program exemplo;

Var

a, b : integer;

c : real;

Begin

Writeln (“Digite dois números do tipo inteiro”);

Readln (a, b);

c:=(a + b)/2;

Writeln (“A média eh:”,c);

End.

INFRAESTRUTURA COMPUTACIONAL

Tempo de compilação X Tempo de execução

Exemplo 1(Execução):

Digite dois números do tipo inteiro

10, 2 <enter>

A média eh: 6

Tempo de compilação X Tempo de execução

Exemplo 2:

Program exemplo;

Var

a, b : integer;

c : real;

Begin

Writeln (“Digite dois números do tipo inteiro”);

Readln (a, b);

c:=a + b/2;

Writeln (“A média eh:”, c);

End.

INFRAESTRUTURA COMPUTACIONAL

Tempo de compilação X Tempo de execução

Exemplo 2(Execução):

Digite dois números do tipo inteiro

10, 2 <enter>

A média eh: 11

FUNDAMENTOS DE SISTEMAS OPERACIONAIS

FUNÇÕES:

=> Tornar a comunicação do homem com a máquina mais inteligente.

=> Possibilitar o uso compartilhado e protegido dos recursos de hardware e de software.

=> Permitir a instalação e configuração dos recursos de hardware e de software.

FUNDAMENTOS DE SISTEMAS OPERACIONAIS

Sistemas operacionais monousuário

CPU somente pode ser utilizada por um único usuário por vez. Possuem processamento centralizado.

EX: MS-DOS

Sistemas operacionais multiusuário

Permite que ocorra mais de um usuário utilizando uma mesma CPU ao “mesmo tempo”, através de terminais ligados ao computador.

Programas e os arquivos de dados estão em um único computador que gerencia o uso dos seus recursos.

EX: Windows, Unix

FUNDAMENTOS DE SISTEMAS OPERACIONAIS

Sistemas operacionais monotarefa

Só conseguem executar uma atividade de cada vez.

EX: MS-DOS

Sistemas operacionais multitarefa

Gerenciam a execução de mais de uma tarefa ao mesmo tempo, seja pela execução simultânea ou pelo compartilhamento do tempo entre as tarefas, dando a impressão de simultaneidade.

EX: Windows, Unix

FUNDAMENTOS DE REDES DE COMPUTADORES

Rede de computador é (são) um (ou mais) módulo(s) processador(es) interligados, por um meio físico (ou não), capaz(es) de realizar desde uma simples troca de arquivos até o compartilhamento de um recurso remoto.

FUNDAMENTOS DE REDES DE COMPUTADORES

Componentes básicos:

⇒ Servidor

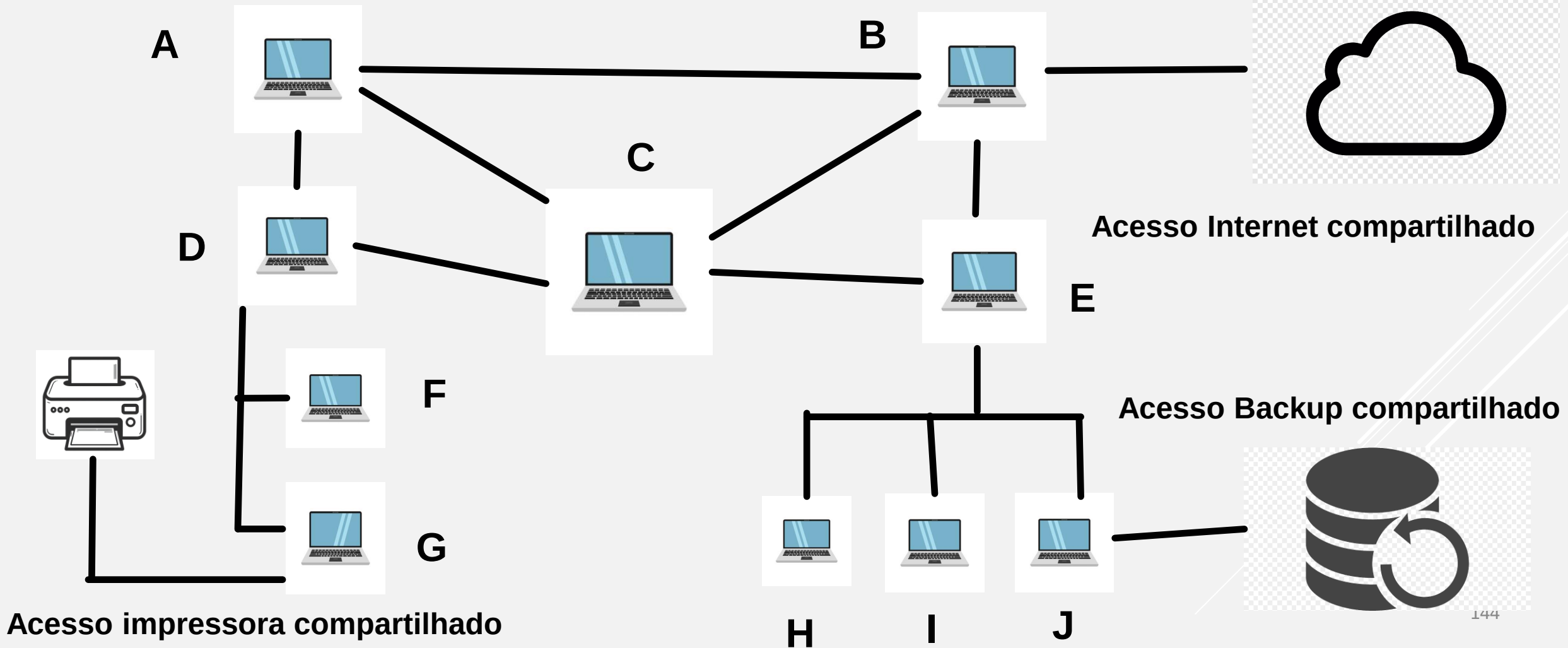
É (são) a(s) máquina(s) que disponibiliza(m) recursos a serem compartilhados pelo ambiente de rede.

⇒ => Terminal

É (são) a(s) máquina(s) que usufrui(em) dos recursos disponibilizados ao ambiente de rede.

INFRAESTRUTURA COMPUTACIONAL

FUNDAMENTOS DE REDES DE COMPUTADORES



FUNDAMENTOS DE REDES DE COMPUTADORES

Componentes básicos:

⇒ Software

É o sistema operacional, que permite a instalação e a configuração dos serviços da rede.

⇒ Cabos

Constituem os meios físicos de interligação entre as máquinas nas redes wired (com cabos). Os mais utilizados são o coaxial, o par trançado e a fibra ótica.

FUNDAMENTOS DE REDES DE COMPUTADORES

Componentes básicos:

⇒ Placas

Realizam a interface entre as máquinas e o meio de comunicação, seja a rede wired ou wireless.

⇒ Router

Realiza o endereçamento e o encaminhamento das mensagens entre os nós (hosts) da rede.

FUNDAMENTOS DE REDES DE COMPUTADORES

Componentes básicos:

⇒ Proxy

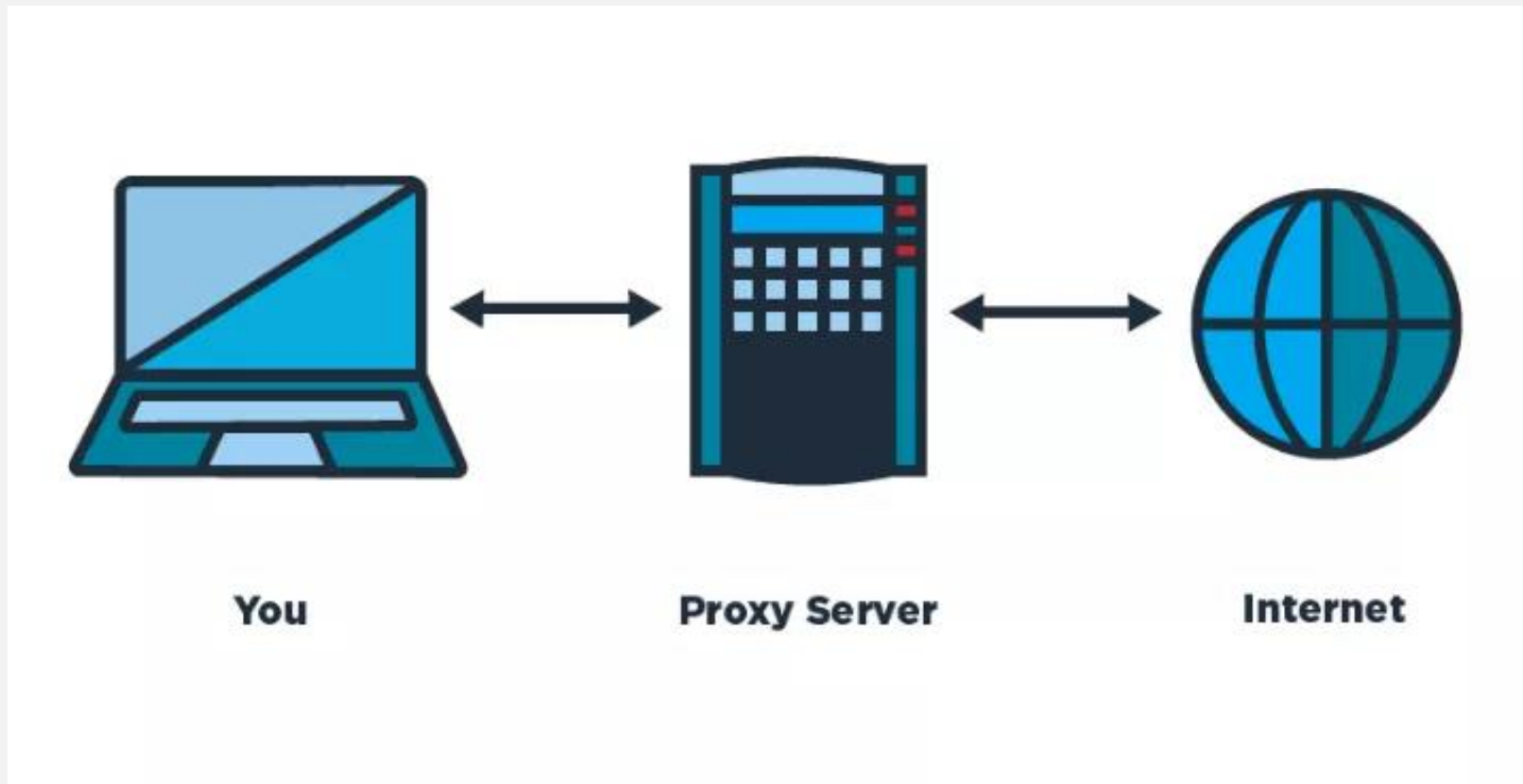
Atua como um filtro entre as informações que saem da rede e o ambiente externo (Internet).

⇒ Firewall

Atua como um filtro entre as informações que entram na rede, provenientes do meio externo (Internet).

FUNDAMENTOS DE REDES DE COMPUTADORES

⇒ Proxy



FUNDAMENTOS DE REDES DE COMPUTADORES

⇒ Firewall

